CAPE PENINSULA UNIVERSITY OF TECHNOLOGY

Department of Electrical, Electronic and Computer Engineering

Marks Administration System

System Design Document

SDN260S — Software Design II · Topic 13 · Group 21

November 2026

This report is the Blackboard system design deliverable. It states the problem, describes the architecture and UML relationships, explains algorithms and advanced Java features, summarises testing, reflects on graduate attributes GA2 and GA7, and records each member's contribution. Source code is submitted as Group21_MarksAdministrationSystem.zip. The Week 12 demo video is recorded separately.

Document section	Brief requirement covered
1. Problem and requirements	Problem definition; Topic 13 feature list
2. Architecture and class design	UML / modularity; data flow
3. Algorithms and advanced features	DSA, recursion, concurrency, serialization
4. Testing and evaluation	Sandbox, edge cases, efficiency, limits
5. Graduate attributes	GA2 and GA7
6. Team contribution	Equal 12.5% breakdown with evidence
7–9. Run guide, challenges, rubric map	README alignment; honesty about open work

1. Problem definition and requirements

At CPUT Bellville, lecturers often keep marks in spreadsheets and email attachments. Totals are recalculated by hand, incomplete assessments disappear into blank cells, and the Head of Department has no single view across modules. Errors are hard to reverse and harder to audit when the semester ends.

The Marks Administration System is a desktop prototype for that workflow. A lecturer logs in, maintains students and subjects, enters assessment scores, imports or exports CSV, and generates student, subject and missing-mark reports. Weighted totals and letter grades are calculated by the system. A Head of Department can do the same work and can also manage user accounts. The console and the Swing GUI share one SQLite database, so a mark entered in either interface survives a restart.

Why it matters

Accurate marks affect progression and graduation. A tool that refuses invalid scores, treats a missing exam as incomplete rather than as a higher average, and keeps an auditable store reduces administrative risk. For the course, the same tool demonstrates OOP, file handling, collections, recursion, concurrency and persistence in one coherent product.

Users and needs

User	Primary needs
Lecturer	Secure login; CRUD for students and subjects; mark entry with validation; CSV template / import / export; reports; missing-entry list.
Head of Department	All lecturer functions plus list / add / edit / delete users; oversight of seeded accounts.
Examiner / marker	Java 17 + Maven only; seeded CPUT data; repeatable console path; automated tests; clear README.

Topic 13 functional checklist

Brief requirement	Implementation status
Capture and manage student marks	Complete — CLI menus and Swing Marks / Student panels
Automatic totals and final grades	Complete — Mark.calculateWeightedTotal and Mark.Grade
CSV uploads and exports	Complete — template, single file, recursive folder import
Flag missing entries	Complete — isMissing flag; missing-marks and subject reports
Summary reports	Complete — student, subject, missing, activity log
Role-based access (lecturer, HoD)	Complete — AuthenticationService; HoD-only user tab / menu
CLI and simple GUI	Complete — App chooses mode 1 or 2

2. System design and architecture

The architecture is layered. Presentation classes (CLIMain, GUIMain and panels) never contain SQL or CSV parsing. Services own the business rules. Persistence is reached only through MarkRepository. Database is the SQLite class that implements that interface. AuthenticationService is the login contract; UserService implements it. This modularity matches the brief's demand for separate classes and clear relationships.

UML-style class relationships

```
App
 ■■■ CLIMain ---------------> AuthenticationService <|.. UserService
 ■      ■■■ MarkService ----> MarkRepository <|.. Database (SQLite JDBC)
 ■      ■■■ ReportService -> MarkService
 ■      ■■■ CSVService ----> MarkService ..> CSVHelper / SnapshotHelper
 ■■■ GUIMain (tabs) -------> same services as CLIMain

MarkService "1" o-- "*" Student "1" o-- "*" Mark
MarkService "1" o-- "*" Subject
UserService ..> PasswordUtils
MarkService ..> CPUTData
```

Data flow for mark entry: the UI builds a Mark map of category → score, calls MarkService.enterMarks, which validates 0–100, aligns categories to the subject, saves through MarkRepository.saveMark inside a transaction, and updates the in-memory student map. CSV import calls the same method, so file rows cannot bypass the rules.

Package layout

Package	Contents
cli / gui	CLIMain; GUIMain, LoginDialog, Dashboard, Student/Subject/Mark/Report/CSV/User panels
services	MarkService, UserService, CSVService, ReportService, AuthenticationService
db	MarkRepository (interface), Database (SQLite schema + CRUD)
models	Student, Subject, Mark, User (+ Role, Grade)
utils	PasswordUtils, CSVHelper, CPUTData, SnapshotHelper

Key design decisions

SQLite was chosen so markers need no XAMPP. Interfaces were introduced so services do not depend on one concrete store. Assessment weights live on Subject, not hard-coded in the grade formula. A settings.seeded flag prevents deleted demo marks from regenerating on every start. Passwords are never stored in plain text.

3. Algorithms, data structures and advanced features

Weighted total algorithm

For each assessment weight w on the subject, a present score s contributes s × (w / 100). The sum of contributions is divided by the sum of all weights (not only the filled ones) and scaled to a percentage. A null score sets isMissing and contributes zero. That stops a student who skipped a 50% exam from passing on the remaining tests alone. Letter grades use fixed thresholds (A+ from 90 down to F below 60). Cost per mark is O(k) for k categories.

```
for each (category, weight) in subject.assessmentWeights:
    totalWeight += weight
    if score[category] is null: missing = true
    else: earned += score[category] * (weight / 100)
weightedTotal = (earned / totalWeight) * 100
letterGrade = threshold(weightedTotal)   // e.g. 83.25 → B+
```

Collections, search and streams

Students and subjects are stored in ConcurrentHashMap for O(1) average look-up by ID. Search filters with the Streams API over the class list (O(n)). Reports compute averages, highest and lowest with stream statistics. Lists and maps are generic for type safety.

Recursion

CSVHelper.findCSVFiles(directory) lists .csv files in a folder, then recursively calls itself for each sub-directory. CSVService.importDirectoryDetailed uses that list. The console menu option “Import Folder Recursively” and the GUI button “Import Folder of CSVs” expose it. Traversal cost is O(f) for f file-system nodes visited.

File I/O, serialization and concurrency

CSV uses OpenCSV. Import matches columns by header name. Background import/export uses CompletableFuture so Swing stays responsive. SnapshotHelper demonstrates object serialization: Student, Subject and Mark implement Serializable; a .ser file is written with ObjectOutputStream and can be read back for demo or recovery. SQLite remains the live store.

Security algorithm

PasswordUtils stores pbkdf2$iterations$salt$hash (120 000 iterations, random 16-byte salt). Verification is constant-time. Older salted SHA-256 hashes still verify so upgrades do not lock accounts out. Role checks gate HoD-only features.

4. Testing and evaluation

The sandbox is CPUTData: seven Bellville programmes, 31 modules, 23 students with nine-digit numbers, and sample marks. Tests and demos use a throw-away SQLite path. Templates live in templates/CPUT_Marks_Template.csv.

Test class	N	Focus
MarkTest	5	Weights, missing scores, category realignment
PasswordUtilsTest	5	PBKDF2, legacy hash, corrupt input
MarkServiceTest	4	Rename categories, validation, seed flag
DatabasePersistenceTest	6	Reload, delete, subject edits, auth
CSVServiceTest	4	Template, import, export round trip, bad header
ReportServiceTest	3	Columns, class statistics, export
MarkPanelTest	1	Headless GUI preload of existing marks
SnapshotHelperTest	3	Object snapshot, recursive find, empty folder

Edge cases: scores outside 0–100, weights ≠ 100%, blank assessments, unknown subject codes, duplicate CSV pairs, and restart persistence. Run mvn test (31 tests expected). Efficiency: ID look-up O(1) average; subject report O(n); folder import O(f + rows). Limitations: console menus are demo-scripted rather than under JUnit; Assessment1…N columns are positional; demo passwords are public for marking and must change in production.

5. Graduate attribute reflection

GA2 — Application of scientific and engineering knowledge

We applied OOP (models, interfaces, encapsulation), collections and streams, recursive directory traversal, CSV file handling, object serialization, multithreaded import, and a defined grading algorithm with missing-mark semantics. Validation lives in one service layer so CLI, GUI and CSV cannot diverge. That is syllabus knowledge used to engineer a working academic tool.

GA7 — Sustainability and impact of engineering activity

One authoritative store and automatic missing flags reduce wasted rework and paper reprints. Embedded SQLite avoids running a permanent database server for demos, cutting setup time and machine load for markers. Background CSV keeps the UI usable on large files. Remaining risks (local file, demo passwords) are documented so a later version can harden them without rewriting the domain rules — an efficient path for future maintenance.

6. Team contribution breakdown

Eight members, four pairs, equal 12.5% each. Detail: docs/WORK_DIVISION.md.

Member	Pair	%	Evidence
Nhlonipho Sishi	Database & Domain	12.5	Database.java, pom.xml
Scelo Mthethwa	Database & Domain	12.5	models/*, CPUTData.java
Lucas Mfulwane	Security & Logic	12.5	PasswordUtils, UserService
Millano Redcliffe	Security & Logic	12.5	MarkService, ReportService
Bopaki Montjane	File / CSV	12.5	CSVHelper, template CSV
Phakamile Tshenyego	File / CSV	12.5	CSVService, ImportSummary
Hussein Akim	UI & QA	12.5	gui/* panels and dialogs
Sinazo Radana	UI & QA	12.5	CLIMain, App, JUnit, docs

Process: pairs owned non-overlapping packages; integration tests and a shared schema contract joined the layers. The project is versioned on GitHub.

7. Build and console demonstration

```
mvn clean test
mvn package
java -Dmarksystem.db=/tmp/demo.db -jar target/marks-administration-system-1.0.0.jar

1 = CLI   2 = GUI
lecturer1 / pass123     hod1 / admin123

Demo: CIV260S, student 253243782, marks 82 / 71 / 90
→ weighted total 83.25, grade B+, then reports + CSV export.
Folder import and .ser snapshot are under CSV Import/Export.
```

8. Challenges faced and remaining work

Fixed during integration: SQLite lock / read-to-write deadlock; assessment rename dropping marks; missing exams inflating averages; CSV columns shifting scores; password-format upgrade; console crashes on bad input; GUI wiping unloaded assessments. Remaining: split large MarkService and Database classes; name-based assessment CSV columns; force password change; more GUI tests; optional shared server. These do not block the prototype hand-in.

9. Rubric coverage summary

Category	M	Addressed in
Problem & requirements	10	Section 1
System design & modularity	15	Section 2
Data structures & algorithms	15	Section 3
Implementation quality	20	Layered code, Javadoc, validation
Concurrency & advanced features	10	Section 3 — threads, generics, streams, serialization
Testing & evaluation	10	Section 4
Documentation & presentation	10	README + this PDF; Week 12 demo pending
GA2 & GA7	5	Section 5
Team collaboration	5	Section 6

Blackboard upload: (1) this PDF; (2) Group21_MarksAdministrationSystem.zip containing source, README, docs/, templates/, pom.xml.